

Aarya Arban

T11 – 05

Assignment No. 2

Aim: Study and design Flutter UI using common widgets and Layouts.

1. Text Widget

The Text widget in Flutter is used to display a string of text with various styling options. It supports font customization, alignment, and decoration. Example:

```
Text(  
  'Welcome to Flutter!',  
  style: TextStyle(fontSize: 20, fontWeight: FontWeight.bold),  
)
```

2. Button Widget

Flutter provides different types of buttons like ElevatedButton, TextButton, and OutlinedButton. These buttons handle user interaction. Example:

```
ElevatedButton( onPressed:  
  () { print('Button Pressed');  
  },  
  child: Text('Login'),  
)
```

3. CheckBox Widget

A CheckBox is used to toggle between checked and unchecked states. It is useful for selecting multiple options. Example:

```
Checkbox(  
  value: isChecked,onChanged: (bool? newValue)
```

```
setState(() {  
    isChecked = newValue!  
  });  
},  
)
```

4. Container Widget

The Container widget is a versatile box model that can hold other widgets and apply padding, margins, decoration, and more. Example:

```
Container(  
  padding: EdgeInsets.all(10),  
  decoration: BoxDecoration(  
    color: Colors.blue,  
    borderRadius: BorderRadius.circular(10),  
  ),  
  child: Text('Hello, Flutter!'),  
)
```

5. MaterialApp Widget

The MaterialApp widget is the root of a Flutter application that follows Material Design guidelines. It provides themes, navigation, and app-wide settings. Example:

```
MaterialApp(  
  home: LoginPage(), debugShowCheckedModeBanner:  
    false,  
)
```

Output:

Login Form

Username

Password

Sign In

[Forgot Password?](#)

Login Form

Username

shreya

Password

Sign In

[Forgot Password?](#)

Conclusion:

In this write-up, we explored fundamental Flutter widgets essential for UI development:

- **Text Widget** for displaying text content.
- **Button Widget** for user interactions.
- **CheckBox Widget** for selection-based inputs.
- **Container Widget** for structuring and styling UI components.
- **MaterialApp Widget** as the root of a Flutter application.

By implementing these widgets in a **Login Page**, we learned how to structure a user-friendly UI while maintaining interactivity and responsiveness.

Understanding these core widgets is crucial for building more complex Flutter applications